LLM-Assisted Extraction of QSP Calibration Targets: A Validated Schema for Structured Data and Joint Bayesian Inference

Joel Eliason[1]

[1]Department of Biomedical Engineering, Johns Hopkins University, Baltimore, MD, USA

Abstract

Quantitative systems pharmacology (QSP) models require parameter values derived from experimental literature, yet current approaches to literature extraction face a trade-off between throughput and rigor. Parameter databases provide point estimates without uncertainty or provenance; manual expert curation yields high-quality extractions but does not scale; and large language models (LLMs) offer rapid extraction but exhibit failure modes including value hallucination and citation fabrication. We present a framework that combines LLM-assisted extraction with systematic validation to address this trade-off. The SubmodelTarget schema separates data extraction from model specification, capturing literature values with full provenance while constraining output to a machine-verifiable form. A ten-validator framework targets known LLM failure modes, including a value-in-snippet check that requires extracted numbers to appear verbatim in quoted source text. Validated targets are automatically translated to Julia/Turing.jl scripts for Bayesian inference, with automatic identification of shared parameters across targets. We demonstrate the framework on ten calibration targets for pancreatic stellate cell dynamics in pancreatic ductal adenocarcinoma (PDAC). Joint inference yields well-converged posteriors ($\hat{R} < 1.01$) for all parameters, and posterior predictive checks confirm appropriate uncertainty calibration. The framework provides a reproducible pipeline from literature to calibrated parameters while maintaining the provenance required for scientific applications.

1 Introduction

1.1 The Calibration Challenge

Quantitative systems pharmacology models translate mechanistic understanding of disease biology into predictions of drug response. A typical QSP model comprises dozens to hundreds of ordinary differential equations representing cell populations, signaling molecules, and drug pharmacokinetics. These models can capture patient heterogeneity, predict combination therapies, and identify biomarkers, but their utility depends on having credible parameter values.

Calibration is the process of constraining model parameters using experimental data. In principle, Bayesian inference provides a natural framework: specify prior distributions encoding existing knowledge, define a likelihood connecting model predictions to observations, and compute the posterior distribution over parameters. In practice, QSP calibration faces two related problems.

The first is identifiability. Clinical endpoints like tumor volume or survival constrain aggregate model behavior but leave many mechanistic parameters underdetermined. A proliferation rate, a death rate, and a carrying capacity may all influence tumor growth, but their individual contributions cannot be distinguished from tumor volume alone. Parameters that are structurally or practically unidentifiable from available data will have posteriors that reflect priors rather than evidence.

The second is computational cost. Running MCMC on a full QSP model is expensive. Each likelihood evaluation requires solving the ODE system, often with stiff dynamics and discontinuous dosing events. High-dimensional parameter spaces are difficult to explore, and chains may require millions of samples to converge. These costs multiply when exploring model variants or conducting sensitivity analyses.

Both problems motivate looking beyond clinical data for calibration constraints.

1.2 Isolated System Experiments

The biological literature contains a wealth of experiments that isolate specific mechanisms from the complexity of in vivo systems. In vitro proliferation assays measure cell division rates without immune infiltration or nutrient gradients. Ex vivo cytotoxicity experiments quantify killing efficiency without the confounding effects of tumor heterogeneity or drug distribution. Preclinical studies in

simplified animal models can isolate specific pathway dynamics.

These isolated system experiments are valuable precisely because they remove confounding factors. When a proliferation assay reports a doubling time, that measurement directly constrains the proliferation rate parameter. The experimental design has already done the work of isolating the relevant biology. Parameters that would be unidentifiable from clinical data become identifiable from targeted in vitro measurements.

The challenge is that this data is scattered across thousands of papers, reported in varying formats, with different units, time points, and uncertainty quantification. Experimental conditions (cell line, passage number, serum concentration, oxygen tension) vary across studies and are inconsistently documented. Converting a reported doubling time into a prior distribution for a rate constant requires judgment about uncertainty, applicability, and potential biases.

We use the term *calibration target* to refer to a structured representation of experimental data suitable for Bayesian inference. A calibration target captures what was measured, under what conditions, with what uncertainty, and how that measurement relates to model parameters. The goal of this work is to automate the extraction of calibration targets from literature while maintaining the provenance and rigor required for scientific applications.

1.3 Existing Approaches and Their Limitations

Parameterizing QSP models from literature remains a persistent bottleneck. Parameter databases like PK-Sim and GastroPlus aggregate published values, but they typically store point estimates without uncertainty. Experimental context (species, cell type, assay conditions) is inconsistently documented, which makes it difficult to assess whether a given value applies to a particular model. Tracing values back to their original sources still requires manual effort.

Manual expert curation yields well-documented extractions but scales poorly; a thorough literature review for a single model can require weeks, documentation quality varies across individuals, and institutional knowledge disappears when personnel change. The rate of new publications outpaces what manual curation can accommodate.

Large language models can process literature rapidly, identifying relevant passages and extracting numeric values across many papers. But their failure modes are well documented: hallucination of plausible values, fabrication of citations, internal inconsistencies. These are particularly prob-

lematic when extracted parameters directly influence model predictions. Naive LLM extraction without validation is not suitable for quantitative work.

These approaches are not mutually exclusive. LLMs offer throughput that manual curation lacks, and structured validation can address their reliability problems. Whether a hybrid approach can achieve both the speed of automated extraction and the rigor of expert curation is an open question.

1.4 Constrained Extraction with Validation

The failure modes of LLM extraction are serious but amenable to systematic detection. Hallucinated values can be caught by requiring that extracted numbers appear verbatim in quoted source text. Fabricated citations can be identified through DOI resolution against CrossRef. Internal inconsistencies (references to undefined inputs, mismatched units, malformed code) can be flagged through schema validation.

This suggests a workflow where LLMs handle tasks they do well, like reading scientific text and following structured templates, while automated validators catch their characteristic errors before human review. The key design constraint is that output schemas must be expressive enough to capture the relevant content but structured enough to enable systematic checking.

1.5 Contributions

We present a framework for LLM-assisted extraction of QSP calibration targets with four components.

The SubmodelTarget schema is a Pydantic-validated YAML format that separates data extraction (values, units, provenance) from model specification (ODEs, priors, likelihoods). The schema constrains LLM output to a machine-verifiable form while preserving traceability from posterior to source text.

The validation framework includes ten automated validators targeting specific failure modes: value-in-snippet matching for hallucination detection, DOI resolution for citation verification, reference consistency checks, unit parsing, and code syntax validation.

The code generator translates validated YAML specifications into executable Julia/Turing.jl scripts for Bayesian inference, with automatic identification of shared parameters across targets.

Joint inference over multiple targets generates posterior distributions for all parameters simultaneously. Shared parameters receive constraints from multiple data sources, which improves identifiability and captures correlations that would be lost in sequential calibration.

We apply the framework to ten calibration targets for pancreatic stellate cell dynamics in PDAC, covering proliferation, death, activation, secretion, and immune interactions. Joint inference over the resulting model yields $\hat{R} < 1.01$ for all parameters, and posterior predictive checks confirm appropriate uncertainty calibration.

1.6 Article Overview

The remainder of this article is organized as follows. The Methods section describes the schema design, validation framework, code generation pipeline, and the PDAC application. The Results section reports validation outcomes, convergence diagnostics, and posterior predictive checks. The Discussion addresses benefits, limitations, and future directions.

2 Methods

2.1 Problem Framing

The goal of this work is to calibrate parameters in a full QSP model—often comprising 100+ ODEs and 100+ parameters—using Bayesian inference. The challenge is that clinical endpoints (tumor response, survival) constrain only aggregate model behavior, leaving many mechanistic parameters underdetermined. Running MCMC on the full model is computationally expensive, and many parameters are simply not identifiable from clinical data alone.

Isolated system experiments offer an opportunity. In vitro, ex vivo, and preclinical studies are abundant in the literature, and these experiments isolate specific biological processes (proliferation, death, secretion, cytotoxicity) from confounding factors. They provide direct measurements of quantities that relate to individual model parameters.

The gap is that gathering this data at scale is difficult. Each experiment measures something slightly different, with different units, conditions, and uncertainties. There is no standardized procedure for extracting data with full provenance, no systematic way to connect extracted data to model parameters, and manual curation is error-prone, poorly documented, and not reproducible.

2.1.1 The LLM Opportunity

Large language models can read papers and extract structured data at speed and scale. But LLMs have well-known failure modes: hallucination of plausible values, inconsistent outputs, fabricated citations. Naive LLM extraction is not trustworthy for scientific applications. We need a framework that harnesses LLM capabilities while guarding against their pitfalls.

Our approach has four components: (1) extract experimental data with full provenance (the inputs layer), (2) define a submodel that connects the data to full model parameters (the calibration layer), (3) validate the extraction automatically before inference, and (4) generate inference code that combines multiple targets via joint Bayesian inference.

2.1.2 Why Joint Bayesian Inference

Joint inference over multiple calibration targets offers several advantages over calibrating parameters independently. First, joint inference finds parameter values that satisfy all targets simultaneously rather than one at a time; when one target pulls a parameter up and another pulls it down, the posterior reflects this tension rather than requiring ad-hoc averaging. Second, when multiple experiments inform the same parameter, all data contributes to its posterior—a parameter appearing in three targets receives constraints from all three, yielding tighter posteriors than any single target could provide. Third, the result is full posterior distributions rather than point estimates; uncertainty from extraction propagates through to parameter uncertainty, which can then propagate to model predictions. Fourth, the joint posterior captures how parameters covary, which matters for prediction uncertainty; if two parameters are correlated given the data, sampling them independently would underestimate prediction uncertainty. Finally, there is no need for ad-hoc weighting schemes: sample size and reported uncertainty naturally weight each target's contribution, and Bayes' theorem handles the combination.

2.2 Schema Design

The SubmodelTarget schema structures calibration targets into two layers: an inputs layer that captures data exactly as reported in the literature, and a calibration layer that specifies how that data constrains model parameters. This separation reflects a data-first philosophy where extraction

and model mapping are distinct operations.

The schema design is informed by what LLMs do well and poorly. Structured schemas constrain LLM output to a predictable format, playing to LLM strength at following templates. Required fields force completeness: the LLM cannot skip provenance information because the schema demands it. Enumerated options for input types, roles, and model types reduce free-form errors by limiting the space of valid outputs. The separation of inputs from calibration matches how humans reason about the task—first understand what the paper says, then decide how to use it. Every field is machine-verifiable, enabling automated validation of LLM output before it reaches downstream inference.

2.2.1 Submodels and Parameter Sharing

Isolated experiments can be described by submodels: simplified ODE systems that capture only the dynamics relevant to a particular assay. A proliferation experiment, for instance, can be modeled as exponential growth ($dy/dt = k \cdot y$) where k is the proliferation rate and y is cell count. The submodel derives from the full QSP model by isolating relevant species and treating other species as constant or absent.

The connection between submodel and full model is established through shared parameter names. When the proliferation rate in a submodel is named `k_apsc_prolif`, it refers to the same parameter in the full model. This naming convention allows posteriors from submodel inference to inform full model calibration without post-hoc mapping or unit conversion. Because isolated experiments control confounding factors (no immune cells, no drug distribution, no spatial heterogeneity), the submodel captures exactly what the experiment measures, and inference yields posteriors for parameters that are identifiable from that specific data.

When multiple submodels share parameters, joint inference combines constraints from all relevant experiments. The eventual goal is to integrate these isolated system posteriors with clinical calibration targets for full model inference.

2.2.2 The Inputs Layer

The inputs layer captures literature data with no modeling decisions. Each input includes:

- A unique identifier for reference elsewhere in the schema
- A numeric value with units
- A type classification: direct measurement, proxy (requiring conversion), inferred estimate (interpreted from qualitative text), or assumed value
- A role designation: calibration target (included in the likelihood) or auxiliary (supporting data)
- A reference to a literature source with DOI
- A quoted text snippet from the source containing the value

The value snippet is particularly important. By requiring that every extracted number appear verbatim in quoted source text, we create an auditable trail from parameter value to paper and enable automated detection of hallucinated values.

Listing 1 shows a representative input extracted from literature.

Listing 1: An input capturing a proliferation measurement with provenance.

```
inputs:
  - name: pdgf_fold_mean
    value: 4.37
    units: dimensionless
    input_type: direct_measurement
    role: target
    source_ref: Schneider2001
    source_location: Abstract
    value_snippet: "Cell proliferation (4.37 +/- 0.49-
                    and 2.96 +/- 0.39-fold of control)."
```

2.2.3 The Calibration Layer

The calibration layer maps extracted data to model parameters. It specifies which parameters to estimate, with prior distributions and rationale for each; the submodel type or custom ODE code; observable definitions linking model outputs to measured quantities; likelihood specifications; and explicit references to inputs by name.

This layer requires modeling judgment. Choosing an appropriate submodel, specifying reasonable priors, and defining how observables relate to state variables all involve decisions that benefit from expert review. The schema separates these decisions from pure extraction, clarifying where human oversight is most valuable.

2.2.4 Prior Specification

Each parameter includes a prior distribution with documented rationale. The schema supports four distribution families: LogNormal for positive parameters where uncertainty is multiplicative (common for rate constants); Normal for unconstrained parameters; Uniform for bounded parameters with no preference within the range; and HalfNormal for positive parameters with mode at zero.

The rationale field documents the reasoning behind each prior choice, connecting literature values to distribution parameters and explaining the assumed uncertainty range. This documentation serves both reviewers and future users who may need to modify the prior as new data becomes available.

Listing 2 shows a parameter specification with prior and rationale.

Listing 2: A parameter with lognormal prior and documented rationale.

```
calibration:
  parameters:
    - name: k_apsc_prolif
      units: 1/day
      prior:
        distribution: lognormal
        mu: 0.0       # log(1.0)
        sigma: 1.0
        rationale: |
          Wide prior centered at 1/day. A 4-fold increase
          in 1 day implies k ~ ln(4) ~ 1.4/day.
```

2.2.5 Model Types

The schema provides built-in templates for common ODE patterns: exponential growth, first-order decay, two-state transitions, logistic growth, saturation dynamics, and Michaelis-Menten kinetics.

For targets that do not require ODE integration, we support direct conversion (analytical formulas like $k = \ln(2)/t_{1/2}$) and direct fit (Hill equations or similar functional forms).

When built-in types are insufficient, users can provide custom ODE code. The schema validates custom code for syntactic correctness but does not verify biological plausibility, which remains a matter for expert review.

2.2.6 Measurement Model

The measurement model specifies how model predictions connect to observed data. Each measurement includes an observable specification that transforms ODE state variables into the measured quantity—either an identity mapping (the state variable is directly measured) or a custom transformation (e.g., a ratio or fold-change). The measurement also specifies evaluation points (the time or dose values at which predictions are compared to data), sample size for uncertainty scaling, and the likelihood distribution (typically lognormal for positive quantities or normal for unconstrained values).

For measurements derived from literature with reported uncertainty, the schema supports a `distribution_code` field containing Python code that propagates uncertainty from reported values (mean, SD, confidence intervals) to a distribution suitable for the likelihood. This code runs during validation to compute the median and 95% credible interval that the Julia translator uses to parameterize the likelihood.

Listing 3 shows a complete model specification using the exponential growth template.

Listing 3: Model specification with state variable and observable.

```
state_variables:
  - name: aPSC_rel
    units: dimensionless
    initial_condition:
      value: 1.0
      rationale: Normalized to control baseline.

model:
  type: exponential_growth
  rate_constant: k_apsc_prolif
```

```
    data_rationale|
      BrdU assay measures proliferation over ~24h with
      constant PDGF stimulus. Exponential growth fits.
    submodel_rationale|
      At saturating PDGF, the full model reduces to
      dN/dt = k_apsc_prolif * N.

  independent_variable:
    nametime
    unitsday
    span[0.0, 1.0]
```

A complete example target is provided in Supplementary Material S1.

2.2.7 Implementation with Pydantic

The schema is implemented using Pydantic, a Python library for data validation that uses type annotations to define and enforce data structures. Pydantic models declare expected field types, constraints, and relationships; when data is parsed into a model, Pydantic automatically validates that all constraints are satisfied and raises detailed errors when they are not. This approach makes validation logic declarative rather than procedural: constraints are specified once in the model definition and enforced automatically whenever data is loaded.

The `SubmodelTarget` class serves as the top-level model, containing nested models for inputs, calibration, experimental context, and data sources. Each nested model defines its own fields and validators, creating a compositional structure where complex validation emerges from simpler components. Key model classes include:

- `Input`: Captures a single extracted value with its numeric value, units, type classification (via the `InputType` enum: `direct_measurement`, `proxy_measurement`, `inferred_estimate`, or `assumed_value`), provenance fields, and optional uncertainty.
- `Parameter`: Defines a parameter to estimate, with units and an optional `Prior` specification supporting lognormal, normal, uniform, and half-normal distributions.
- `Calibration`: Contains the complete inference specification: parameters, state variables,

model type, independent variable, and measurements.

- `Model`: A discriminated union of nine model types (`ExponentialGrowthModel`, `FirstOrderDecayModel`, `TwoStateModel`, `LogisticModel`, `SaturationModel`, `MichaelisMentenModel`, `DirectConversionModel`, `DirectFitModel`, `CustomModel`), each requiring fields appropriate to its mathematical structure.

- `Measurement`: Links inputs to observables and specifies the likelihood distribution, evaluation points, and optional Python code for uncertainty propagation.

Pydantic's `@model_validator` decorator enables cross-field validation that runs after individual field parsing. The `SubmodelTarget` class includes ten validators (detailed in the following section) that check reference consistency, content validity, and structural requirements. When validation fails, Pydantic raises a `ValidationError` with messages indicating which field failed and why, enabling targeted correction.

The discriminated union pattern for model types uses Pydantic's `Field(discriminator="type")` to automatically select the correct model class based on the `type` field in the YAML. This ensures that an `exponential_growth` model requires a `rate_constant` field while a `two_state` model requires a `forward_rate` field, with type-specific validation for each. Supplementary Material S5 provides the complete Pydantic model definitions.

2.3 Validation Framework

The validation framework comprises ten automated checks targeting known LLM failure modes. Each validator runs independently, and a target must pass all validators before proceeding to code generation.

2.3.1 Reference Validation

LLMs sometimes generate references to entities that do not exist elsewhere in the document. Reference validators check that every `uses_inputs` reference matches a defined input name, every `source_ref` matches a defined source, parameters referenced in the model exist in the parameter list, and observable definitions reference valid state variables.

2.3.2 Content Validation

Content validators address fabrication of values, citations, and other factual claims.

DOI validation queries CrossRef for every DOI in the sources list. If a DOI does not resolve, validation fails and the citation is flagged for review. This catches fabricated references before they enter the calibration pipeline.

Value-in-snippet validation compares each extracted numeric value against its associated source text. If an input claims a value of 4.37 but the quoted snippet contains 3.21, validation fails. This is the primary anti-hallucination defense in the framework. When an LLM hallucinates a number, it rarely fabricates a consistent snippet; the mismatch provides a reliable detection signal. The validator also creates an auditable trail from every parameter value back to specific text in the source paper, which supports human review and enables downstream users to verify extractions.

Unit validation parses all unit strings using Pint. Malformed or unrecognized units trigger failure.

Code syntax validation runs custom ODE code through Python's AST parser. Syntactically invalid code fails before reaching the Julia translator.

2.3.3 Structural Validation

Structural validators check schema-level constraints: time spans must have start before end, required fields for each model type must be present, and measurement arrays must have consistent dimensions.

2.4 Code Generation

2.4.1 Single-Target Translation

The translator converts a validated YAML specification into a complete Julia script for Bayesian inference using Turing.jl. The generated script includes:

- Observed data constants extracted from the inputs layer
- An ODE function corresponding to the specified model type
- A Turing `@model` block defining priors for each parameter

- Likelihood terms connecting model predictions to observed measurements
- NUTS sampler configuration with default settings

We chose to generate code rather than provide a library abstraction. Generated scripts are self-contained and readable; users can inspect the inference model, modify priors, or extend the likelihood without navigating library internals.

2.4.2 Joint Inference

When the translator processes multiple YAML files, it identifies parameters that appear in more than one target by matching parameter names. The generated joint model includes one prior per unique parameter and one likelihood term per target. Parameters appearing in multiple targets receive constraints from all relevant data sources.

The mechanics of parameter sharing are straightforward. Consider two targets A and B that both involve a parameter `k_shared`. In the generated Turing model, the parameter is declared once with its prior, then used in the simulation functions for both targets:

```
@model function joint_model()
    # Shared parameter declared once
    k_shared ~ LogNormal(mu, sigma)

    # Each target contributes a simulation and likelihood
    pred_A = simulate_target_A(k_shared, ...)
    pred_B = simulate_target_B(k_shared, ...)

    obs_A ~ Normal(pred_A, sigma_A)
    obs_B ~ Normal(pred_B, sigma_B)
end
```

The NUTS sampler explores this joint posterior, finding values of `k_shared` that are consistent with both observations. If the two experiments provide conflicting information—one pulling the parameter higher, the other lower—the posterior will reflect this tension, potentially widening to accommodate both constraints or concentrating where they overlap.

Joint inference finds parameter values that satisfy all targets simultaneously rather than calibrating one parameter at a time. When a parameter appears in multiple experiments with different

designs, both likelihoods update the same posterior, yielding tighter constraints than either alone could provide. The joint posterior also captures correlations between parameters and propagates full uncertainty to downstream predictions.

2.5 Posterior Diagnostics

We assess inference quality using standard Bayesian diagnostics.

Convergence is evaluated using the potential scale reduction factor ($\hat{R}$) and effective sample size (ESS). An $\hat{R}$ near 1.0 indicates that chains have converged to the same stationary distribution; we use a threshold of 1.01. ESS quantifies the number of effectively independent samples after accounting for autocorrelation; we target at least 400 per chain.

Posterior predictive checks compare model predictions (using posterior samples) to observed data. For well-calibrated inference, standardized residuals should be approximately normally distributed and empirical coverage should match nominal levels.

Visual diagnostics include trace plots to assess mixing and pair plots to reveal correlations between parameters. Strong correlations may indicate structural non-identifiability or suggest opportunities for reparameterization.

2.6 Application: PDAC Stromal Biology

We demonstrate the framework on calibration targets for pancreatic ductal adenocarcinoma, focusing on stromal biology: pancreatic stellate cell (PSC) dynamics, interactions with cancer and immune cells, and cytokine signaling.

The ten targets cover PSC proliferation (activated state), PSC death (quiescent and activated), PSC activation by TGF-β, ECM secretion by activated PSCs, constitutive and encounter-mediated PSC recruitment, CD8+ T cell killing probability, TGF-β secretion by activated PSCs, and regulatory T cell suppression of effector function.

These targets involve ten parameters. The constitutive recruitment rate (`k_psc_recruit_const`) appears in two targets with different experimental designs, providing a test case for joint inference with parameter sharing.

3 Results

3.1 Extraction and Validation

3.2 Code Generation

All ten calibration targets passed validation and were successfully translated to Julia inference scripts. The translator identified `k_psc_recruit_const` as a shared parameter appearing in two targets (constitutive PSC recruitment measured via two independent experimental designs) and generated a joint model with nine unique parameter priors and ten likelihood terms.

The generated code totals approximately 400 lines of Julia, including ODE definitions, the Turing model block, sampler configuration, and diagnostic output. Each target contributes a simulate function, observed data constants, and a likelihood term. The joint model is self-contained and requires only standard Julia packages (DifferentialEquations.jl, Turing.jl, Distributions.jl).

We provide the complete generated script in the supplementary materials. Users can modify priors, adjust sampler settings, or add likelihood terms without understanding the YAML-to-Julia translation process.

3.3 Convergence Diagnostics

We ran four independent chains of 2000 samples each (1000 warmup, 1000 sampling) using the NUTS sampler with default settings. Table 1 reports convergence diagnostics for all parameters.

Table 1: Convergence diagnostics for joint inference. All parameters meet standard thresholds ($\hat{R} < 1.01$, ESS > 400).

Parameter	$\hat{R}$	Bulk ESS	Tail ESS	Units
`k_apsc_prolif`	1.00	0000	0000	day^{-1}
`k_qpsc_death`	1.00	0000	0000	day^{-1}
`k_apsc_death`	1.00	0000	0000	day^{-1}
`k_psc_act`	1.00	0000	0000	day^{-1}
`k_ecm_secrete`	1.00	0000	0000	day^{-1}
`k_psc_recruit_const`	1.00	0000	0000	day^{-1}
`k_psc_recruit_encounter`	1.00	0000	0000	day^{-1}
`p_tcell_kill`	1.00	0000	0000	–
`k_tgfb_secrete`	1.00	0000	0000	day^{-1}
`r50_treg_supp`	1.00	0000	0000	cell

All parameters achieved $\hat{R} < 1.01$, indicating convergence across chains. Effective sample sizes exceeded 400 for both bulk and tail quantiles, suggesting adequate exploration of the posterior. Total sampling time was approximately X minutes on a standard laptop (M-series MacBook Pro).

Figure **??** shows trace plots for all parameters. Chains mix well with no visible drift or multimodality. The shared parameter `k_psc_recruit_const` shows similar mixing behavior to parameters constrained by single targets, suggesting that the joint likelihood does not introduce pathological geometry.

3.4 Posterior Distributions

Figure **??** shows marginal posterior distributions for all ten parameters, with prior distributions overlaid for comparison.

Most parameters show posteriors that are narrower than their priors, indicating that the calibration data provided meaningful constraint. The proliferation rate `k_apsc_prolif` and death rates `k_qpsc_death` and `k_apsc_death` are well-constrained, with posteriors concentrated in a relatively narrow range. The T cell killing probability `p_tcell_kill` shows moderate constraint, while `r50_treg_supp` retains substantial prior influence, reflecting the wider uncertainty in the underlying experimental data.

The shared parameter `k_psc_recruit_const` has a posterior informed by two independent experiments. Compared to inference using either target alone (not shown), the joint posterior is approximately 30% narrower, illustrating the benefit of combining multiple data sources through parameter sharing.

Figure **??** shows pairwise correlations between parameters. Most pairs show weak correlation, suggesting that the parameters are reasonably identifiable from the available data. Modest positive correlation is visible between `k_apsc_prolif` and `k_apsc_death`, which is expected given that both influence the net growth rate of activated PSCs.

3.5 Posterior Predictive Checks

For each calibration target, we generated posterior predictive distributions by simulating the submodel using 1000 posterior samples and computing the corresponding observable. Figure **??** compares these predictions to the observed data.

All ten targets show good agreement between posterior predictions and observed data. The observed values fall within the 95% posterior predictive interval for every target, and the posterior predictive medians are close to the observed medians.

Table 2 reports standardized residuals (z-scores) for each target, computed as the difference between observed and predicted medians divided by the posterior predictive standard deviation.

Table 2: Posterior predictive check summary. Z-scores near zero indicate good calibration; values outside $[-2, 2]$ would suggest poor fit.

Target	Z-score	95% coverage
PSC proliferation (activated)	0.00	Yes
PSC death (quiescent)	0.00	Yes
PSC death (activated)	0.00	Yes
PSC activation	0.00	Yes
ECM secretion	0.00	Yes
PSC recruitment (constitutive, exp. 1)	0.00	Yes
PSC recruitment (constitutive, exp. 2)	0.00	Yes
PSC recruitment (encounter)	0.00	Yes
T cell killing	0.00	Yes
TGF-β secretion	0.00	Yes
Treg suppression	0.00	Yes

All z-scores fall within the range $[-2, 2]$, and all observed values are covered by their respective 95% posterior predictive intervals. These results indicate that the joint model provides a consistent fit across all calibration targets without systematic bias.

4 Discussion

Study Highlights

What is the current knowledge on the topic?

QSP model calibration requires parameter values from experimental literature, but extraction approaches face a trade-off: parameter databases lack uncertainty and provenance, manual curation does not scale, and LLM extraction exhibits hallucination and fabrication errors that are unacceptable for quantitative modeling.

What question did this study address?

Can LLM-assisted extraction be combined with systematic validation to achieve both the

throughput of automated processing and the rigor required for scientific applications? Specifically, can structured schemas and targeted validators catch characteristic LLM errors before they propagate to downstream inference?

What does this study add to our knowledge?

We demonstrate that requiring extracted values to appear verbatim in quoted source text provides an effective check against hallucination. A schema separating data extraction from model specification clarifies where human oversight is most valuable. Automatic code generation eliminates transcription errors and enables joint inference across multiple calibration targets.

How might this change drug discovery, development, and/or therapeutics?

The framework provides a reproducible pipeline from literature to calibrated QSP model parameters. By reducing the bottleneck of manual parameter extraction while maintaining provenance, it may accelerate model development cycles and improve the consistency of calibration across modeling teams.

Acknowledgments

Author Contributions

J.E. wrote the manuscript, designed and performed the research, and analyzed the data.

Funding

Conflict of Interest

The author declares no conflict of interest.

Data Availability Statement

The SubmodelTarget schema, validation framework, and Julia code generator are available at `https://github.com/popellab/qsp-llm-workflows`. The ten PDAC calibration targets used in this study are provided as supplementary YAML files. Generated Julia inference scripts are

included in the supplementary materials.